

Orders & Payments APIs

Complete documentation for order processing, payment handling, and ticket management.

📌 Overview

The Orders & Payments system provides:

- **Order Management:** Create and manage event ticket orders
- **Payment Processing:** Secure payment handling via Stripe integration
- **Ticket Generation:** Automatic ticket creation and delivery
- **Payment Analytics:** Comprehensive financial reporting
- **Refund Processing:** Automated refund capabilities

🔑 Testing & Regional Compliance

Live vs Test Keys Configuration

Environment Setup:

- **Development:** Automatically uses test keys regardless of configuration
- **Staging/Production:** Uses live keys when `USE_LIVE_KEYS=true`
- **Force Test Mode:** Set `VITE_FORCE_TEST_MODE=true` to always use test keys

Test Cards

When testing payment functionality, use region-appropriate test cards:

India-Compliant Test Cards (Recommended):

- 4000003560000008 - Visa India (domestic transaction)
- 5200007840000022 - Mastercard UAE (for international testing)

Live Account Configuration:

- **Current Setup:** Live Stripe account with India compliance mode
- **Regulatory Status:** `live-india` compliance mode active
- **Key Format:** Live keys corrected from `rk_live_` to `sk_live_` format

Important Notes:

- ⚠️ **Live Account:** Now using live Stripe keys - real charges will occur
- 🇮🇳 **India Compliance:** Live account configured for Indian regulatory requirements
- 🔑 **Development Mode:** Test Payment method remains recommended for development
- 🔑 **Key Validation:** Automatic key format checking and environment warnings
- 📖 **Documentation:** [Stripe India Export Regulations \(https://stripe.com/docs/india-exports\)](https://stripe.com/docs/india-exports)

Error Handling

Common regulatory compliance errors:

```
{
  "error": {
    "type": "invalid_request_error",
    "message": "As per Indian regulations, only registered Indian businesses can accept international payments."
  }
}
```

Recommended Approach:

1. Implement fallback to test payment methods
2. Show user-friendly error messages
3. Provide alternative payment options
4. Include links to regulatory documentation

Production Deployment Checklist

Before deploying to production:

- 1. ☒ Verify live Stripe keys are correctly formatted (sk_live_ not rk_live_)
- 2. ☒ Set PAYMENT_ENVIRONMENT=production and USE_LIVE_KEYS=true
- 3. ☒ Test payment flow with small amounts in staging environment
- 4. ☒ Confirm regulatory compliance for target markets
- 5. ☒ Enable monitoring for payment failures and regulatory errors
- 6. ☒ Verify webhook endpoints are configured for live events
- 7. ☒ Test fallback mechanisms work with live account restrictions

Environment Variables for Production:

```
# Backend (.env)
PAYMENT_ENVIRONMENT=production
USE_LIVE_KEYS=true
STRIPE_SECRET_KEY=sk_live_... (corrected format)
STRIPE_COMPLIANCE_MODE=live-india

# Frontend (.env)
VITE_PAYMENT_ENVIRONMENT=production
VITE_USE_LIVE_KEYS=true
VITE_FORCE_TEST_MODE=false
VITE_STRIPE_COMPLIANCE_MODE=live-india
```

 Order Management

POST /api/orders

Create a new order for event tickets.

Authentication: Required (Customer role)

Headers:

```
Authorization: Bearer <access_token>
Content-Type: application/json
```

Request Body:

```
{
  "items": [
    {
      "eventId": "68a155c40214f9e8ddf64aff",
      "scheduleDate": "2024-07-15T18:00:00Z",
      "quantity": 2
    }
  ],
  "billingAddress": {
    "firstName": "John",
    "lastName": "Doe",
    "email": "john@example.com",
    "phone": "+1234567890",
    "address": "123 Main Street",
    "city": "Dubai",
    "state": "Dubai",
    "zipCode": "12345",
    "country": "UAE"
  },
  "notes": "Special requirements or requests",
  "couponCode": "DISCOUNT10",
  "affiliateCode": "PARTNER123"
}
```

Response:

```
{
  "success": true,
  "message": "Order created successfully",
  "data": {
    "order": {
      "_id": "order_id_123",
      "orderNumber": "ORD-1647890123456",
      "customerId": "customer_id",
      "items": [
        {
          "eventId": {
            "_id": "68a155c40214f9e8ddf64aff",
            "title": "Summer Music Festival",
            "vendorId": "vendor_id"
          },
          "scheduleDate": "2024-07-15T18:00:00Z",
          "quantity": 2,
          "unitPrice": 299,
          "totalPrice": 598
        }
      ],
      "subtotal": 598,
      "discounts": {
        "couponDiscount": 59.8,
        "affiliateDiscount": 0
      },
      "taxes": 53.82,
      "total": 591.02,
      "currency": "AED",
      "status": "pending",
      "paymentStatus": "pending",
      "billingAddress": {...},
      "createdAt": "2024-03-16T10:30:00.000Z",
      "expiresAt": "2024-03-16T11:00:00.000Z"
    }
  }
}
```

GET /api/orders

Retrieve user's orders with filtering and pagination.

Authentication: Required (Customer role)

Headers:

```
Authorization: Bearer <access_token>
```

Query Parameters:

- **status** (string): Filter by order status ("pending", "confirmed", "cancelled", "refunded")
- **paymentStatus** (string): Filter by payment status ("pending", "paid", "failed", "refunded")
- **sortBy** (string): Sort field ("createdAt", "total", "status")
- **sortOrder** (string): Sort order ("asc", "desc", default: "desc")
- **page** (number): Page number (default: 1)
- **limit** (number): Items per page (default: 10, max: 50)
- **dateFrom** (string): Filter orders from date (ISO format)
- **dateTo** (string): Filter orders to date (ISO format)

Example Request:

```
curl -X GET "http://localhost:5000/api/orders?status=confirmed&limit=5" \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN"
```

Response:

```
{
  "success": true,
  "message": "Orders retrieved successfully",
  "data": {
    "orders": [
      {
        "_id": "order_id",
        "orderNumber": "ORD-1647890123456",
        "items": [
          {
            "eventId": {
              "title": "Summer Music Festival",
              "images": ["https://cloudinary.com/festival.jpg"],
              "dateSchedule": [{
                "date": "2024-07-15T18:00:00Z"
              }]
            },
            "quantity": 2,
            "totalPrice": 598
          }
        ],
        "total": 591.02,
        "currency": "AED",
        "status": "confirmed",
        "paymentStatus": "paid",
        "createdAt": "2024-03-16T10:30:00.000Z"
      }
    ],
    "pagination": {
      "currentPage": 1,
      "totalPages": 3,
      "totalOrders": 12,
      "hasNextPage": true,
      "hasPrevPage": false,
      "limit": 5
    }
  }
}
```

GET /api/orders/:id

Get detailed information for a specific order.

Authentication: Required (Order owner only)

Headers:

```
Authorization: Bearer <access_token>
```

Path Parameters:

- **id** (string): Order MongoDB ObjectId

Response:

```
{
  "success": true,
  "message": "Order details retrieved successfully",
  "data": {
    "order": {
      "_id": "order_id",
      "orderNumber": "ORD-1647890123456",
      "customerId": {
        "_id": "customer_id",
        "firstName": "John",
        "lastName": "Doe",
        "email": "john@example.com"
      },
      "items": [
        {

```

```

    "eventId": {
      "_id": "event_id",
      "title": "Summer Music Festival",
      "description": "Amazing music festival...",
      "location": {
        "city": "Dubai",
        "address": "Marina Beach"
      },
      "vendorId": {
        "firstName": "Ahmed",
        "lastName": "Events",
        "email": "ahmed@events.com"
      }
    },
    "scheduleDate": "2024-07-15T18:00:00Z",
    "quantity": 2,
    "unitPrice": 299,
    "totalPrice": 598
  }
],
"pricing": {
  "subtotal": 598,
  "discounts": {
    "couponCode": "DISCOUNT10",
    "couponDiscount": 59.8,
    "affiliateCode": "PARTNER123",
    "affiliateDiscount": 0
  },
  "taxes": {
    "vatRate": 0.05,
    "vatAmount": 26.91,
    "serviceFeeRate": 0.05,
    "serviceFeeAmount": 26.91
  },
  "total": 591.02
},
"currency": "AED",
"status": "confirmed",
"paymentStatus": "paid",
"paymentDetails": {
  "paymentMethod": "stripe",
  "paymentIntentId": "pi_1234567890",
  "paidAt": "2024-03-16T10:35:00.000Z"
},
"billingAddress": {...},
"tickets": [
  {
    "_id": "ticket_id_1",
    "ticketNumber": "TKT-SMFEST-001",
    "qrCode": "https://cloudinary.com/qr-codes/ticket1.png",
    "status": "active"
  },
  {
    "_id": "ticket_id_2",
    "ticketNumber": "TKT-SMFEST-002",
    "qrCode": "https://cloudinary.com/qr-codes/ticket2.png",
    "status": "active"
  }
],
"createdAt": "2024-03-16T10:30:00.000Z",
"updatedAt": "2024-03-16T10:35:00.000Z"
}
}
}

```

PUT /api/orders/:id/cancel

Cancel an order before payment or within cancellation window.

Authentication: Required (Order owner only)

Headers:

```
Authorization: Bearer <access_token>
Content-Type: application/json
```

Path Parameters:

- `id` (string): Order MongoDB ObjectId

Request Body:

```
{
  "reason": "Changed my mind about attending",
  "notes": "Will book again for next year"
}
```

Response:

```
{
  "success": true,
  "message": "Order cancelled successfully",
  "data": {
    "order": {
      "_id": "order_id",
      "orderNumber": "ORD-1647890123456",
      "status": "cancelled",
      "cancelledAt": "2024-03-16T11:00:00.000Z",
      "cancellationReason": "Changed my mind about attending"
    },
    "refund": {
      "amount": 591.02,
      "currency": "AED",
      "status": "processing",
      "estimatedArrival": "2024-03-23T11:00:00.000Z"
    }
  }
}
```

Payment Processing

POST /api/payments/create-intent

Create a Stripe payment intent for order processing.

Authentication: Required (Customer role)

Headers:

```
Authorization: Bearer <access_token>
Content-Type: application/json
```

Request Body:

```
{
  "orderId": "order_id_123",
  "savePaymentMethod": true,
  "metadata": {
    "eventName": "Summer Music Festival",
    "customerNote": "VIP upgrade requested"
  }
}
```

Response:

```
{
  "success": true,
  "message": "Payment intent created successfully",
  "data": {
    "clientSecret": "pi_1234567890_secret_abcdefg",
    "paymentIntentId": "pi_1234567890",
    "amount": 59102,
    "currency": "aed",
    "order": {
      "_id": "order_id_123",
      "total": 591.02,
      "status": "pending_payment"
    }
  }
}
```

GET /api/payments/config

Get Stripe public configuration for frontend integration.

Authentication: Not required

Example Request:

```
curl http://localhost:5000/api/payments/config
```

Response:

```
{
  "success": true,
  "message": "Payment configuration retrieved successfully",
  "data": {
    "publishableKey": "pk_test_1234567890abcdefg",
    "country": "AE",
    "supportedCurrencies": ["AED", "USD", "EUR"],
    "paymentMethods": ["card", "apple_pay", "google_pay"],
    "appearance": {
      "theme": "stripe",
      "variables": {
        "colorPrimary": "#0570de"
      }
    }
  }
}
```

POST /api/payments/confirm

Confirm payment after successful client-side processing.

Authentication: Required (Customer role)

Headers:

```
Authorization: Bearer <access_token>
Content-Type: application/json
```

Request Body:

```
{
  "paymentIntentId": "pi_1234567890",
  "paymentMethodId": "pm_1234567890"
}
```

Response:

```
{
  "success": true,
  "message": "Payment confirmed successfully",
  "data": {
    "order": {
      "_id": "order_id",
      "status": "confirmed",
      "paymentStatus": "paid"
    },
    "tickets": [
      {
        "ticketNumber": "TKT-SMFEST-001",
        "qrCode": "https://cloudinary.com/qr-codes/ticket1.png"
      }
    ],
    "emailSent": true,
    "smsNotificationSent": true
  }
}
```

POST /api/payments/cancel

Cancel a payment intent before completion.

Authentication: Required (Customer role)

Headers:

```
Authorization: Bearer <access_token>
Content-Type: application/json
```

Request Body:

```
{
  "paymentIntentId": "pi_1234567890",
  "reason": "Customer changed mind"
}
```

GET /api/payments/payment-methods

Retrieve customer's saved payment methods.

Authentication: Required (Customer role)

Headers:

```
Authorization: Bearer <access_token>
```

Response:

```
{
  "success": true,
  "message": "Payment methods retrieved successfully",
  "data": {
    "paymentMethods": [
      {
        "id": "pm_1234567890",
        "type": "card",
        "card": {
          "brand": "visa",
          "last4": "0008",
          "expMonth": 12,
          "expYear": 2025
        },
        "isDefault": true,
        "createdAt": "2024-03-01T10:00:00.000Z"
      }
    ]
  }
}
```


DELETE /api/payments/payment-methods/:id

Remove a saved payment method.

Authentication: Required (Customer role)

Headers:

```
Authorization: Bearer <access_token>
```

Path Parameters:

- `id` (string): Payment method ID

Response:

```
{
  "success": true,
  "message": "Payment method removed successfully"
}
```

POST /api/payments/webhook

Stripe webhook endpoint for payment status updates.

Authentication: Stripe signature verification (not JWT)

Note: This endpoint is used exclusively by Stripe to notify payment status changes. It uses Stripe's webhook signature verification instead of JWT authentication.

Ticket Management

GET /api/tickets

Retrieve user's tickets with filtering options.

Authentication: Required (Customer role)

Headers:

```
Authorization: Bearer <access_token>
```

Query Parameters:

- `status` (string): Filter by ticket status ("active", "used", "cancelled", "expired")
- `eventId` (string): Filter by specific event
- `upcoming` (boolean): Show only upcoming events ("true"/"false")
- `page` (number): Page number (default: 1)
- `limit` (number): Items per page (default: 10, max: 50)
- `sortBy` (string): Sort field ("createdAt", "eventDate", "ticketNumber")
- `sortOrder` (string): Sort order ("asc", "desc")

Example Request:

```
curl -X GET "http://localhost:5000/api/tickets?status=active&upcoming=true" \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN"
```

Response:

```
{
  "success": true,
  "message": "Tickets retrieved successfully",
  "data": {
    "tickets": [
      {
        "_id": "ticket_id",
        "ticketNumber": "TKT-SMFEST-001",
        "orderId": "order_id",
        "eventId": {
          "_id": "event_id",
          "title": "Summer Music Festival",
          "location": {
            "city": "Dubai",
            "address": "Marina Beach"
          },
          "images": ["https://cloudinary.com/festival.jpg"],
          "dateSchedule": [{
            "date": "2024-07-15T18:00:00Z"
          }]
        },
        "qrCode": "https://cloudinary.com/qr-codes/ticket1.png",
        "qrCodeData": "TKT-SMFEST-001|2024-07-15T18:00:00Z|customer_id",
        "status": "active",
        "purchaseDate": "2024-03-16T10:35:00.000Z",
        "checkInDetails": {
          "isCheckedIn": false,
          "checkInTime": null,
          "scanCount": 0
        }
      }
    ],
    "pagination": {...}
  }
}
```

GET /api/tickets/id

Get detailed information for a specific ticket.

Authentication: Required (Ticket owner only)

Headers:

```
Authorization: Bearer <access_token>
```

Path Parameters:

- `id` (string): Ticket MongoDB ObjectId

Response:

```
{
  "success": true,
  "message": "Ticket details retrieved successfully",
  "data": {
    "ticket": {
      "_id": "ticket_id",
      "ticketNumber": "TKT-SMFEST-001",
      "orderId": {
        "_id": "order_id",
        "orderNumber": "ORD-1647890123456"
      },
    },
    "eventId": {
      "_id": "event_id",
      "title": "Summer Music Festival",
      "description": "The biggest music event...",
      "location": {
        "coordinates": {
          "lat": 25.232,
          "lng": 55.2515
        },
        "city": "Dubai",
        "address": "Marina Beach, Marina Walk"
      },
      "dateSchedule": [{
        "date": "2024-07-15T18:00:00Z",
        "_id": "schedule_id"
      }],
      "faqs": [...],
      "vendorId": {
        "firstName": "Ahmed",
        "lastName": "Events",
        "phone": "+971501234567"
      }
    },
    "customerId": {
      "firstName": "John",
      "lastName": "Doe",
      "email": "john@example.com"
    },
    "qrCode": "https://cloudinary.com/qr-codes/ticket1.png",
    "qrCodeData": "encrypted_ticket_data",
    "status": "active",
    "purchaseDate": "2024-03-16T10:35:00.000Z",
    "checkInDetails": {
      "isCheckedIn": false,
      "checkInTime": null,
      "checkInBy": null,
      "checkInLocation": null,
      "scanCount": 0
    },
    "transferHistory": [],
    "validUntil": "2024-07-16T02:00:00Z"
  }
}
```

POST /api/tickets/id/transfer

Transfer ticket to another user via email.

Authentication: Required (Ticket owner only)

Headers:

```
Authorization: Bearer <access_token>
Content-Type: application/json
```

Path Parameters:

- id (string): Ticket MongoDB ObjectId

Request Body:

```
{
  "toEmail": "recipient@example.com",
  "message": "Enjoy the event! Can't wait to hear about it.",
  "transferType": "gift"
}
```

Response:

```
{
  "success": true,
  "message": "Ticket transfer initiated successfully",
  "data": {
    "transfer": {
      "_id": "transfer_id",
      "ticketId": "ticket_id",
      "fromCustomerId": "sender_id",
      "toEmail": "recipient@example.com",
      "status": "pending",
      "transferCode": "TRF-ABC123DEF",
      "expiresAt": "2024-03-23T10:35:00.000Z",
      "message": "Enjoy the event! Can't wait to hear about it."
    },
    "emailSent": true
  }
}
```

POST /api/tickets/id/resend

Resend ticket via email or SMS.

Authentication: Required (Ticket owner only)

Headers:

```
Authorization: Bearer <access_token>
Content-Type: application/json
```

Path Parameters:

- `id` (string): Ticket MongoDB ObjectId

Request Body:

```
{
  "method": "email",
  "recipient": "john@example.com"
}
```

Available methods: "email", "sms"

Response:

```
{
  "success": true,
  "message": "Ticket sent successfully via email",
  "data": {
    "deliveryMethod": "email",
    "recipient": "john@example.com",
    "sentAt": "2024-03-16T15:30:00.000Z"
  }
}
```

Vendor Order Management

GET /api/orders/vendor/my-orders

Retrieve orders for vendor's events.

Authentication: Required (Vendor role)

Headers:

Authorization: Bearer <access_token>

Query Parameters:

- `eventId` (string): Filter by specific event
- `status` (string): Filter by order status
- `paymentStatus` (string): Filter by payment status
- `page` (number): Page number (default: 1)
- `limit` (number): Items per page (default: 12, max: 100)

Response:

```
{
  "success": true,
  "message": "Vendor orders retrieved successfully",
  "data": {
    "orders": [
      {
        "_id": "order_id",
        "orderNumber": "ORD-1647890123456",
        "customerId": {
          "firstName": "John",
          "lastName": "Doe",
          "email": "john@example.com"
        },
        "eventId": {
          "title": "My Music Festival",
          "dateSchedule": [...]
        },
        "total": 591.02,
        "currency": "AED",
        "status": "confirmed",
        "paymentStatus": "paid",
        "createdAt": "2024-03-16T10:30:00.000Z"
      }
    ],
    "summary": {
      "totalOrders": 156,
      "totalRevenue": 98750.50,
      "averageOrderValue": 633.02
    },
    "pagination": {...}
  }
}
```

 Admin Order Management

GET /api/orders/admin/all

Retrieve all orders with comprehensive filtering.

Authentication: Required (Admin role)

Headers:

Authorization: Bearer <access_token>

Query Parameters:

- `status` (string): Filter by order status
- `paymentStatus` (string): Filter by payment status
- `vendorId` (string): Filter by vendor
- `eventId` (string): Filter by event
- `customerId` (string): Filter by customer
- `dateFrom` (string): Filter from date
- `dateTo` (string): Filter to date
- `sortBy` (string): Sort field ("createdAt", "total", "status", "paymentStatus")
- `sortOrder` (string): Sort order ("asc", "desc")

- `page` (number): Page number (default: 1)
- `limit` (number): Items per page (default: 20, max: 100)

GET /api/orders/admin/analytics

Get comprehensive order analytics for admin dashboard.

Authentication: Required (Admin role)

Headers:

Authorization: Bearer <access_token>

Query Parameters:

- `period` (number): Analysis period in days (1-365, default: 30)
- `groupBy` (string): Group analytics by ("day", "week", "month")

Response:

```
{
  "success": true,
  "message": "Order analytics retrieved successfully",
  "data": {
    "summary": {
      "totalOrders": 2456,
      "totalRevenue": 1250000.75,
      "averageOrderValue": 508.95,
      "conversionRate": 0.125,
      "refundRate": 0.045
    },
    "trends": {
      "orderGrowth": 0.15,
      "revenueGrowth": 0.22,
      "period": "30_days"
    },
    "breakdown": {
      "byStatus": {
        "confirmed": 2201,
        "pending": 125,
        "cancelled": 95,
        "refunded": 35
      },
      "byPaymentMethod": {
        "stripe": 2205,
        "bank_transfer": 156,
        "cash": 95
      },
      "topVendors": [
        {
          "vendorId": "vendor_1",
          "vendorName": "Ahmed Events",
          "orders": 345,
          "revenue": 175250.00
        }
      ]
    },
    "timeSeriesData": [
      {
        "date": "2024-03-01",
        "orders": 89,
        "revenue": 45125.50
      }
    ]
  }
}
```

POST /api/payments/refund

Process refund for completed payments.

Authentication: Required (Admin role)

Headers:

```
Authorization: Bearer <access_token>
Content-Type: application/json
```

Request Body:

```
{
  "orderId": "order_id_123",
  "amount": 591.02,
  "reason": "Customer request - event cancelled",
  "refundType": "full",
  "notifyCustomer": true
}
```

Response:

```
{
  "success": true,
  "message": "Refund processed successfully",
  "data": {
    "refund": {
      "_id": "refund_id",
      "orderId": "order_id_123",
      "amount": 591.02,
      "currency": "AED",
      "reason": "Customer request - event cancelled",
      "status": "processing",
      "stripeRefundId": "re_1234567890",
      "estimatedArrival": "2024-03-23T10:30:00.000Z",
      "processedBy": "admin_id",
      "processedAt": "2024-03-16T14:30:00.000Z"
    },
    "order": {
      "status": "refunded",
      "refundedAt": "2024-03-16T14:30:00.000Z"
    }
  }
}
```

Implementation Examples

Frontend Order Creation Flow

```

// Complete order creation with payment processing
const createOrderWithPayment = async (orderData) => {
  try {
    // Step 1: Create order
    const orderResponse = await fetch('/api/orders', {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${accessToken}`,
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(orderData)
    });

    const order = await orderResponse.json();

    // Step 2: Create payment intent
    const paymentResponse = await fetch('/api/payments/create-intent', {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${accessToken}`,
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        orderId: order.data.order._id
      })
    });

    const payment = await paymentResponse.json();

    // Step 3: Process payment with Stripe Elements
    const { error } = await stripe.confirmPayment({
      elements,
      clientSecret: payment.data.clientSecret,
      confirmParams: {
        return_url: `${window.location.origin}/order-complete`
      }
    });

    if (error) {
      throw new Error(error.message);
    }

    return order.data.order;
  } catch (error) {
    console.error('Order creation failed:', error);
    throw error;
  }
};

```

Ticket Display Component


```
// React component for displaying user tickets
const TicketCard = ({ ticket }) => {
  const downloadQR = async (ticketId) => {
    const response = await fetch(`/api/tickets/${ticketId}/qr`, {
      headers: {
        'Authorization': `Bearer ${accessToken}`
      }
    });

    const blob = await response.blob();
    const url = URL.createObjectURL(blob);

    const a = document.createElement('a');
    a.href = url;
    a.download = `ticket-${ticket.ticketNumber}.png`;
    a.click();

    URL.revokeObjectURL(url);
  };

  return (
    <div className="ticket-card">
      <img src={ticket.qrCode} alt="QR Code" />
      <h3>{ticket.eventId.title}</h3>
      <p>Ticket: {ticket.ticketNumber}</p>
      <p>Date: {new Date(ticket.eventId.dateSchedule[0].date).toLocaleDateString()}</p>
      <button onClick={() => downloadQR(ticket._id)}>
        Download QR Code
      </button>
    </div>
  );
};
```

Related Documentation:

- [Authentication → \(./authentication.md\)](#) - User authentication and role management
- [Events → \(events.md\)](#) - Event creation and management
- [File Uploads → \(file-uploads.md\)](#) - Ticket and receipt document management
- [Admin APIs → \(admin-apis.md\)](#) - Administrative order and payment management